

# Organização do Projeto

Explicação da estrutura de pastas do backend do sistema de Gerenciamento de Empresas

## 1. Objetivo

Este documento explica como o backend do projeto está organizado, descrevendo a finalidade de cada pasta para que qualquer membro do grupo saiba onde encontrar ou onde colocar um novo arquivo.

## 2. Visão geral da raiz

| Item                                                        | Descrição                                                                                    |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>.vscode/</code>                                       | Configurações do editor VS Code compartilhadas entre a equipe (debug e settings do projeto). |
| <code>app/</code>                                           | Código-fonte principal da aplicação Flask (rotas, regras de negócio, infraestrutura).        |
| <code>database/</code>                                      | Schema do banco de dados e scripts de migração e seed.                                       |
| <code>docs/</code>                                          | Documentação do projeto (arquitetura, API, banco de dados, Docker, este próprio guia).       |
| <code>scripts/</code>                                       | Scripts utilitários de linha de comando para automatizar tarefas do projeto.                 |
| <code>tests/</code>                                         | Testes automatizados da aplicação.                                                           |
| <code>.env</code> / <code>.env.example</code>               | Variáveis de ambiente (a real não vai para o Git; a <code>.example</code> serve de modelo).  |
| <code>.gitignore</code> / <code>.dockerignore</code>        | Arquivos e pastas que não devem ser versionados ou copiados para a imagem Docker.            |
| <code>Makefile</code>                                       | Atalhos de comandos usados no dia a dia (ex.: subir containers, rodar migrations).           |
| <code>pyproject.toml</code> / <code>requirements.txt</code> | Definição das dependências Python do projeto.                                                |
| <code>run.py</code>                                         | Ponto de entrada usado para iniciar a aplicação.                                             |

## 3. Pasta `app/` (código da aplicação)

É onde fica todo o código-fonte da API. Dividida em quatro grandes áreas:

| Pasta                            | O que deve conter                                                                                                                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>app/main.py</code>         | Criação e configuração da aplicação Flask (application factory).                                                                                                                                                                                      |
| <code>app/config/</code>         | Configurações da aplicação: variáveis de ambiente ( <code>environment.py</code> ), extensões do Flask ( <code>extensions.py</code> ) e <code>debug.py</code> .                                                                                        |
| <code>app/infrastructure/</code> | Tudo que conecta a aplicação a recursos externos: banco de dados ( <code>database/</code> ), serviços externos como e-mail ( <code>email/</code> ), <code>redis/</code> , <code>stripe/</code> , <code>twilio/</code> , <code>whatsapp/</code> , etc. |
| <code>app/modules/</code>        | Módulos de domínio do sistema (ex.: empresas, usuários, permissões). Cada funcionalidade do sistema deve ter um módulo próprio.                                                                                                                       |
| <code>app/shared/</code>         | Código compartilhado entre módulos: exceções customizadas ( <code>exceptions/</code> ), middlewares ( <code>middleware/</code> ), <code>utils/</code> , <code>validators/</code> , etc.                                                               |

## 4. Pasta `database/`

| Pasta/Arquivo               | O que deve conter                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------------------------|
| <b>database/schema.sql</b>  | Definição completa e atual do schema do banco de dados.                                              |
| <b>database/migrations/</b> | Uma migration por arquivo, numerada em ordem (000, 001, 002...), cada uma criando ou alterando uma p |
| <b>database/seeds/</b>      | Scripts para popular o banco com dados iniciais/teste.                                               |

## 5. Pasta docs/

| Pasta/Arquivo                | O que deve conter                                                                            |
|------------------------------|----------------------------------------------------------------------------------------------|
| <b>docs/api/</b>             | Documentação da API: exceções (exceptions/) e padrão de respostas (response/).               |
| <b>docs/architecture/</b>    | Documentos sobre decisões e organização de arquitetura do sistema, incluindo este documento. |
| <b>docs/database/</b>        | Diagramas e documentação do banco de dados (ex.: arquivo .drawio do ERD).                    |
| <b>docs/development/</b>     | Guias para o time de desenvolvimento, como o padrão de commits.                              |
| <b>docs/docker/</b>          | Arquivos de documentação/configuração do ambiente Docker (Dockerfile, docker-compose.yml).   |
| <b>docs/project-tree.txt</b> | Snapshot da árvore de pastas do projeto, usado como referência.                              |

## 6. Pasta scripts/

| Pasta/Arquivo                 | O que deve conter                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>scripts/cli/</b>           | Comandos de linha de comando customizados do projeto (ex.: criar migration, criar módulo novo, criar see |
| <b>scripts/cli/templates/</b> | Templates usados pelos comandos de CLI para gerar código novo automaticamente (ex.: esqueleto de um      |
| <b>scripts/database/</b>      | Scripts para migrar, resetar e popular (seed) o banco de dados.                                          |
| <b>scripts/maintenance/</b>   | Scripts de manutenção geral do projeto.                                                                  |

## 7. Pasta tests/

Contém os testes automatizados, organizados espelhando a estrutura da aplicação: **tests/core/** para testes do núcleo da aplicação, **tests/database/** para testes de conexão e transações com o banco, **tests/modules/** para testes dos módulos de domínio, **tests/routes/** para testes das rotas da API e **tests/shared/** para testes do código compartilhado (ex.: segurança).

## 8. Onde este documento fica no repositório

docs/architecture/project-structure.pdf